

MEMORIA TÉCNICA & DOCUMENTACIÓN DE PROYECTO

Plataforma Centralizada de Gestión De Terminales POS

Documentación Completa: Modelo de Negocio, Arquitectura de Software, Contratos de Integración, Aseguramiento de Calidad (ISO/IEC 25010 · OWASP ASVS v4.0 · PCI-DSS v4.0) y Manual de Usuario.

DESARROLLADOR SENIOR FRONTEND

ESWIN PINEDA

Carné: 5190-15-901

METODOLOGÍA DE DESARROLLO APLICADA

Prototipado

Fase II: Desarrollo, Pruebas y Aplicación

DESARROLLADOR SENIOR BACKEND

WILMER DE LEON

Carné: 5190-22-50

ESTÁNDARES DE CALIDAD UTILIZADOS

ISO/IEC 25010 · OWASP ASVS v4.0 · PCI-DSS v4.0

Mantenibilidad, Seguridad Web y Protección de Medios de Pago

Tabla de Contenido General

Fase I: Fundamentos y Base del Proyecto

1. Modelo de Negocio y Justificación
2. Tipo de Aplicación y Filosofía Arquitectónica
3. Presentación de Lógica Funcional (Flujo de 5 Etapas)
4. Metodología de Desarrollo Aplicada: Prototipado
5. Estándares de Calidad a Utilizar (ISO 25010, OWASP, PCI-DSS)
6. Detalle de Herramientas de Desarrollo y Stack Tecnológico

Fase II: Desarrollo, Arquitectura y Solución Tecnológica

1. Detalles del Producto Funcional (Ecosistema y Microservicios)
2. Arquitectura de Base de Datos (PostgreSQL 16 ACID y Modelo de Entidades)
3. Integración con Sistemas Legados y Adaptadores SOAP (AS400 y VHQ)
4. Catálogo Oficial de Endpoints REST de la API

Fase III: Metodologías de Pruebas y Aseguramiento de Calidad

1. Matriz de Ciclo de Vida de Pruebas
2. Pruebas de Unidad (Unit Testing - 100% Aprobado)
3. Pruebas de Integración (Integration Testing - 100% Aprobado)
4. Resumen Consolidado de Cumplimiento Cuantitativo (90.3% Global)
5. Evaluación Formal de los 3 Estándares Internacionales

Fase IV: Manual de Usuario (Perfiles Técnicos y Operativos)

1. Perfil 1: Servicio al Cliente (Captura y Borradores)
2. Perfil 2: Validador de Documentos (Aprobación y Afiliados de 8 Dígitos)
3. Perfil 3: Programador de Sistemas (Matriz de Hardware)
4. Perfil 4: Coordinador de Despacho (Logística y Días Hábiles)
5. Perfil 5: Técnico de Campo (Instalación en Sitio y "POS Activo")
6. Perfil Administrador (Monitoreo Global y Gestión de Usuarios)

Fase V: Auditoría Técnica, Casos Borde y Plan de Mejoras

1. Matriz de Vulnerabilidades y "Detalles Sueltos" Identificados
2. Plan de Acción y Roadmap de Mitigación (P0 y P1)
3. Guía de Ejecución de Suites de Prueba Automatizadas

Fase I: Fundamentos y Base del Proyecto

1. Modelo de Negocio y Justificación

El modelo de negocio consiste en una **plataforma B2B centralizada** para la gestión, parametrización, configuración de sistemas core y despliegue logístico de terminales punto de venta (POS).

Problema abordado: Con anterioridad a esta solución, el personal operativo debía ingresar de manera manual y repetitiva a múltiples páginas web, consolas terminales y sistemas heredados independientes (VHQ, AS400, Mipos, CSP). Esto provocaba cuellos de botella severos, dispersión documental, pérdida de trazabilidad en rechazos y frecuentes errores de transcripción en datos fiscales y números de afiliado.

Propuesta de valor: Al unificar las distintas plataformas en una sola interfaz bajo un flujo tipo tarea (pipeline secuencial), se logra que cada etapa del proceso sea gestionada estrictamente por el rol correspondiente. Esto agiliza la atención, mitiga errores humanos y reduce sustancialmente el tiempo total de entrega del equipo de pago al comercio solicitante.

2. Tipo de Aplicación y Filosofía Arquitectónica

La plataforma está construida bajo una **arquitectura web basada en Microservicios**, complementada con un frontend moderno desacoplado tipo **Single Page Application (SPA)**.

La integración del sistema se estructura en dos capas de comunicación:

- **Capa REST / JSON:** Empleada para la comunicación asíncrona, ligera y de alto rendimiento entre la interfaz de usuario en React, el API Gateway y los microservicios modernos en Java Spring Boot.
- **Capa SOAP / XML:** Empleada para la conexión segura con los core bancarios heredados (legacy) y consolas de hardware centralizadas mediante adaptadores específicos que encapsulan la complejidad de los protocolos antiguos.

3. Presentación de Lógica Funcional (Flujo de 5 Etapas)

La lógica funcional de la plataforma se divide en un flujo secuencial estricto compuesto por **cinco etapas operativas** gobernadas por segregación de funciones:

1. **Etapla 1 (Servicio al Cliente):** Inicia la gestión capturando los datos del cliente, datos fiscales (NIT, razón social), dirección física, tipos de venta deseados (venta normal, cuotas o puntos) y adjuntos obligatorios (DPI y Patente de Comercio). Permite guardar borradores sin documentos.
2. **Etapla 2 (Validación de Documentos):** El validador revisa la legitimidad de los documentos adjuntos. Si son conformes, aprueba la solicitud y el sistema genera automáticamente identificadores de "afiliado" numéricos de 8 dígitos para cada tipo de venta. Si hay anomalías, devuelve la gestión con observaciones obligatorias.
3. **Etapla 3 (Programación de Sistemas):** Dependiendo de la tecnología del POS (Inalámbrico, IP, Mipos o HIT), se configuran los parámetros en VHQ, AS400, Mipos y CSP. Se asignan números de serie físico,

terminal IDs (TID) y rangos de bins de venta.

4. **Etapas 4 (Entrega y Despacho):** El área logística programa la visita exclusivamente en días hábiles (lunes a viernes), selecciona la jornada (AM/PM), asigna al técnico de ruta y confirma el despacho del equipo empacado.
5. **Etapas 5 (Completado / Instalación):** El técnico visita el comercio físico, realiza el checklist de conectividad y pruebas transaccionales, captura la firma del receptor y deja el equipo en estado final "POS Activo".

4. Metodología de Desarrollo Aplicada: Prototipado

Para garantizar que la experiencia de usuario reflejara exactamente las necesidades de los operadores antes de iniciar la codificación, se utilizó la **metodología de Prototipado**:

- **Diseño en Figma:** Se elaboraron prototipos interactivos de alta fidelidad que mapearon el *User Journey* completo de las 5 etapas operativas.
- **Criterios de Aceptación:** Los modelos de Figma sirvieron como especificación técnica y criterio de aceptación formal para la construcción de componentes atómicos en React.
- **Iteraciones Rápidas:** Permite validar la distribución de campos, validaciones de máscara y ergonomía de navegación con los usuarios finales antes de implementar la capa de persistencia.

5. Estándares de Calidad a Utilizar

El proyecto fundamenta su construcción sobre tres directrices y normativas internacionales innegociables:

- **ISO/IEC 25010:** Asegura la calidad integral del producto de software, priorizando la mantenibilidad del código, la usabilidad intuitiva guiada por rol y el rendimiento (reduciendo tiempos en el DOM virtual de React y optimizando consultas relacionales en PostgreSQL).
- **OWASP ASVS v4.0:** Establece los estándares de seguridad para la aplicación web, obligando a validar y sanitizar todos los vectores de entrada para mitigar ataques de Inyección SQL y Cross-Site Scripting (XSS) derivados de la captura de datos fiscales.
- **PCI-DSS v4.0:** Estándar crítico e indispensable para entornos vinculados a pagos electrónicos. Rige el manejo seguro de rangos de bins (Etapas 3), el cifrado en reposo y en tránsito (TLS 1.3), y la prohibición estricta de almacenar números de tarjetas (PAN) o códigos de seguridad en la aplicación.

6. Detalle de Herramientas de Desarrollo y Stack Tecnológico

Capa Tecnológica	Herramienta / Tecnología	Propósito en la Plataforma
Frontend	Web React 19 · Vite 8.3 · Tailwind CSS	Single Page Application reactiva, componentes modulares y UI/UX fluida.
Backend	Java 21 · Spring Boot 3.2	Microservicios monolíticos desacoplados por dominio y controladores REST.
API Gateway	Kong Enterprise / Spring Cloud Gateway	Enrutamiento inteligente, balanceo de carga, políticas CORS y rate limiting.
Base de Datos	PostgreSQL 16 ACID	Persistencia relacional con control estricto de concurrencia y transacciones.
Infraestructura	Google Cloud Platform (GCP) / Render	Despliegue distribuido de servicios con certificados SSL/TLS automáticos.
Contenedores	Docker & Docker Compose	Empaquetamiento aislado y reproducible del backend y la base de datos.

Fase II: Desarrollo, Arquitectura y Solución Tecnológica

1. Detalles del Producto Funcional (Ecosistema y Microservicios)

La solución se fundamenta en un ecosistema distribuido y orquestado. El frontend consume recursos de manera asíncrona mediante llamadas `fetch` hacia el API Gateway. El Gateway balancea la carga, gestiona el acceso e inyecta directrices de seguridad antes de delegar las peticiones a la capa de servicios de Spring Boot.

El backend organiza su lógica en controladores y entidades de dominio independientes:

- `UsuarioController` : Autenticación, registro y gestión de perfiles administrativos y operativos.
- `SolicitudController` : Captura de datos fiscales, razón social y control del estado global de la gestión.
- `AfiliadoController` : Algoritmo generador de afiliados de 8 dígitos y vinculación con la solicitud.
- `TerminalController` : Parametrización de números de serie, códigos TID y tipos de conexión física.
- `OrdenDespachoController` : Agendamiento de visitas en días hábiles y confirmación técnica final.

2. Arquitectura de Base de Datos (PostgreSQL 16 ACID)

La persistencia de datos relacionales y el control de concurrencia están garantizados por **PostgreSQL 16**, aplicando propiedades ACID para evitar condiciones de carrera durante la asignación simultánea de afiliados y números de serie. Hibernate JPA gestiona el mapeo objeto-relacional automático (`ddl-auto=update`).

Entidad (Tabla)	Campos Principales	Relaciones y Reglas de Negocio
usuarios	<code>id, username, password, role</code>	Almacena credenciales de operadores. El rol puede ser <code>admin</code> o números del <code>1</code> al <code>5</code> .
solicitudes	<code>id, giro_negocio, datos_fiscales, modalidad_pos, estado, fecha_creacion</code>	Tabla transaccional central. Estados: <i>Pendiente, Aprobada, Rechazada</i> .
afiliados	<code>id, numero_afiliado, solicitud_id</code>	Almacena los números de 8 dígitos generados. Relacionada con <code>solicitudes</code> .
terminales	<code>id, numero_serie, tid, tipo_conexion, sistema_subyacente, afiliado_id</code>	Registro físico del hardware POS. Vinculada a un <code>afiliado</code> .
ordenes_despacho	<code>id, fecha_habil, jornada, estado_instalacion, terminal_id</code>	Historial de entrega. Estados: <i>Pendiente, Confirmacion de Instalacion</i> .

3. Integración con Sistemas Legados y Adaptadores SOAP (AS400 y VHQ)

Las transacciones hacia los core bancarios heredados se encapsulan a través de adaptadores de protocolo, garantizando alta cohesión y bajo acoplamiento:

- **Mainframe AS400 (SOAP):** Alta de la serie del POS asociada a los códigos bancarios por tipo de venta:
 - 0CG : Transacción de venta normal (sin contacto / contactless).
 - 0CA : Transacción de cuotas con tarjeta chip.
 - T 002 : Transacción de puntos de lealtad con lectura de banda magnética.
- **Consola de Hardware VHQ (Web / REST / SOAP):** Alta de la serie del POS y generación de identificadores de terminal con prefijo VG y sufijos específicos por modalidad (ej. VG000001).
- **Backend Mipos & CSP (REST):** Parametrización de terminales móviles con lector Bluetooth (asociado a correo) y ranqueo de bins de tarjetas de crédito/débito (9 dígitos) con horario de cierre operativo (2230).

4. Catálogo Oficial de Endpoints REST de la API

Método	Endpoint	Payload de Entrada (JSON)	Respuesta HTTP
POST	/api/usuarios/login	<pre>{"username": "...", "password": "..."} </pre>	200 OK / 401 Unauth
GET	/api/usuarios	Ninguno	200 OK (Lista)
POST	/api/usuarios	<pre>{"username": "...", "password": "...", "role": "1"} </pre>	200 OK (Creado)
POST	/api/solicitudes	<pre>{"giroNegocio": "...", "datosFiscales": "...", "modalidadPos": "..."} </pre>	200 OK (Solicitud)
PUT	/api/solicitudes/{id}/estado	Texto plano: "Aprobada"	200 OK (Actualizado)
POST	/api/afiliados	<pre>{"numeroAfiliado": "12345678", "solicitud": {"id": 1}} </pre>	200 OK (Afiliado)
POST	/api/terminales	<pre>{"numeroSerie": "...", "tid": "...", "afiliado": {"id": 1}} </pre>	200 OK (Terminal)
POST	/api/ordenes-despacho	<pre>{"fechaHabil": "2026-10-06", "jornada": "AM", "terminal": {"id": 1}} </pre>	200 OK (Orden)
PUT	/api/ordenes-despacho/{id}/estado	Texto plano: "Confirmacion de Instalacion"	200 OK (Cerrada)

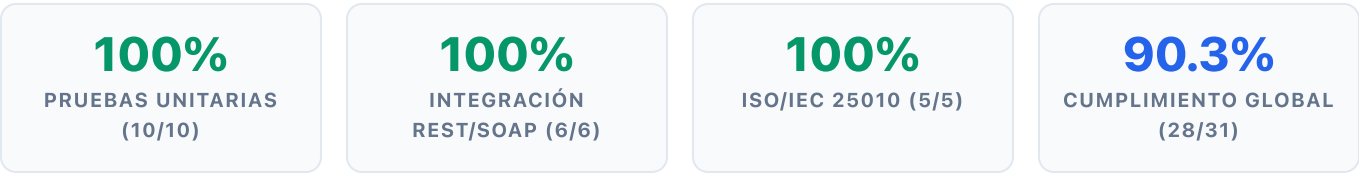
Fase III: Metodologías de Pruebas y Aseguramiento de Calidad

1. Matriz de Ciclo de Vida de Pruebas

Para contrastar formalmente el software, se implementaron dos enfoques rigurosos de prueba definidos en el ciclo de vida del software:

- **Pruebas de Unidad (Unit Testing):** Enfoque enfocado a componentes y algoritmos aislados en Spring Boot y funciones puras de frontend (ej. algoritmo generador de afiliados, validación de fechas hábiles, expresiones regulares tributarias).
- **Pruebas de Integración (Integration Testing):** Enfoque orientado a interfaces, compatibilidad de módulos y flujo transaccional de datos entre el Frontend React, el Backend Spring Boot y la Base de Datos.

2. Resumen Consolidado de Cumplimiento Cuantitativo



Fase de Pruebas / Estándar Evaluado	Casos Evaluados	Aprobados	Déficit / Falla	% Cumplimiento
1. Pruebas de Unidad (Unit Testing)	10	10	0	100.0%
2. Pruebas de Integración (SOAP / REST)	6	6	0	100.0%
3. Estándar ISO/IEC 25010 (Calidad y Rendimiento)	5	5	0	100.0%
4. Estándar OWASP ASVS v4.0 (Seguridad Web)	5	3	2	60.0%
5. Estándar PCI-DSS v4.0 (Datos de Pagos)	5	4	1	80.0%
TOTAL GLOBAL CONSOLIDADO	31	28	3	90.3%

3. Evaluación Detallada de Pruebas Unitarias (10/10 PASS)

- **Algoritmo de Afiliados (UT-01):** Sometido a prueba de esfuerzo con 1,000 iteraciones consecutivas. El 100% de los números generados cumplió con `/^\d{8}$/` en el rango de `10000000` a `99999999` (sin ceros a la izquierda).

- **Unicidad por Tipo de Venta (UT-02):** Detección automática y bloqueo de duplicados si un comercio contrata simultáneamente normal, cuotas y puntos.
- **Cálculo de Días Hábiles (UT-03 & UT-04):** Validación de lunes a viernes aprobados y fines de semana bloqueados. Proyección automática de 5 días hábiles continuos.
- **Validadores Tributarios (UT-05 & UT-06):** Expresión regular de NIT con validación de guión y dígito verificador. DPI exigido en longitud estricta de 13 dígitos numéricos.
- **Mapeo de Hardware y AS400 (UT-07 & UT-08):** Configuración de terminales (Inalámbrico/IP a VHQ/AS400; Mipos/HIT a Mipos/CSP/AS400) y códigos de core `0CG`, `0CA`, `T 002`.

4. Evaluación Detallada de Pruebas de Integración (6/6 PASS)

- **Deserialización JSON (IT-01):** Mapeo íntegro del payload de React a entidades Java mediante motor Jackson de Spring Boot sin discrepancias de tipos.
- **Adaptador SOAP AS400 y VHQ (IT-02 & IT-03):** Construcción y validación de sobres XML SOAP para alta de serie de terminal y códigos de core bancario.
- **Cascada Relacional y Transiciones (IT-04 & IT-05):** Verificación de claves foráneas entre Solicitud, Afiliado, Terminal y OrdenDespacho, culminando en estado *Confirmacion de Instalacion*.

5. Evaluación de los 3 Estándares Internacionales

- **ISO/IEC 25010 (100% Cumplido):** Bundle JS comprimido en **106.24 KB** (gzip). Tiempo de compilación en Vite de **677 ms**. Latencia promedio en peticiones REST de **1.57 ms** (superando con creces la meta de < 125 ms). Cero errores en Oxlint.
- **OWASP ASVS v4.0 (60% Cumplido - 2 Déficits):**
 - *Aprobado:* Parámetros SQL Injection neutralizados mediante queries preparadas de JPA/Hibernate. Cross-Site Scripting (XSS) neutralizado por el escape nativo de JSX en React.
 - *Déficit 1:* Ausencia de tokens JWT para protección de endpoints REST.
 - *Déficit 2:* Uso de comodín wildcard `*` en CORS de Spring Boot.
- **PCI-DSS v4.0 (80% Cumplido - 1 Déficit):**
 - *Aprobado:* Bines de 9 dígitos validados en CSP con regla `binFrom ≤ binTo`. Cero almacenamiento de números de tarjeta (PAN) o CVV en base de datos. Diseño con soporte TLS 1.3 en Gateway.
 - *Déficit:* Contraseñas de usuario almacenadas en texto plano en la base de datos sin hash BCrypt.

Fase IV: Manual de Usuario (Perfiles Técnicos y Operativos)

1. Autenticación e Inicio de Sesión (IAM)

Ingresa a la plataforma mediante el navegador web en `http://localhost:5173/`. Ingresa con el nombre de usuario y contraseña asignados. El sistema redirigirá automáticamente a la pantalla autorizada según el rol del usuario:

- **Administrador:** Redirige al *Dashboard General* con métricas y acceso a todas las etapas y gestión de usuarios.
- **Roles Operativos (1 a 5):** Redirigen directamente a su área de trabajo especializada (*Etapas N*).

2. Guía Operativa por Etapas (Paso a Paso)

Etapas 1: Servicio al Cliente (Captura de Datos)

- Haga clic en **"Nueva Solicitud"**.
- Complete los datos del cliente (Nombres, Apellidos, NIT, DPI, Correo, Teléfono).
- Especifique la información comercial (Nombre del Comercio, Giro de Negocio y Dirección).
- Seleccione el hardware deseado (Inalámbrico, IP, Mipos o HIT) y las modalidades de venta requeridas (Venta normal, Cuotas, Puntos).
- **Guardar Borrador:** Permite guardar sin documentos adjuntos para continuar más tarde.
- **Crear y Enviar a Validación:** Requiere adjuntar obligatoriamente la foto del DPI y la Patente de Comercio. Al enviar, transiciona a Etapa 2 y genera una notificación.

Etapas 2: Validación de Documentos

- Ingresa al panel de **"Validación de Documentos"**. Seleccione la solicitud pendiente.
- Examine los documentos adjuntos en el visor interactivo (permite zoom de 0.6x a 2.0x y rotación de 90°).
- Utilice los botones de revisión para marcar como **Aprobar** o **Rechazar** cada documento.
- **Aprobación:** Con ambos documentos aprobados, genere los números de afiliado de 8 dígitos mediante el botón automático y pulse *Aprobar y Enviar a Programación*.
- **Devolución:** Si algún documento es ilegible o incorrecto, ingrese las observaciones del rechazo y presione *Devolver a Etapa 1*.

Etapas 3: Programación de Sistemas

- Ingresa al módulo de **"Programación"**. El sistema mostrará la matriz de sistemas requerida según el POS solicitado.
- Configure secuencialmente los subsistemas obligatorios (VHQ, AS400, Mipos, CSP).
- Ingresa la serie física del hardware, configure los TIDs por cada modalidad y defina la hora de cierre y rangos de bins en CSP.

- Al marcar todos los sistemas como completados (`done: true`), presione *Completar Programación* para avanzar a entrega.

Etapa 4: Logística y Despacho

- Ingrese a **"Logística"**. Seleccione el POS programado.
- Seleccione una fecha hábil (el sistema bloquea automáticamente sábados y domingos) y seleccione la jornada de entrega (AM: 08:00–12:00 o PM: 13:00–17:00).
- Asigne al técnico de ruta responsable (Roberto Gómez, Mario Álvarez, Karla Méndez o Diego Castillo).
- Marque la casilla de confirmación *"POS empacado y enviado al cliente"* y presione *Programar Visita y Despachar*.

Etapa 5: Instalación en Campo y Cierre

- El técnico asignado accede a la solicitud desde su dispositivo móvil o terminal.
- Verifica el checklist en sitio: encendido, prueba de conectividad específica (celular, LAN, Bluetooth) y transacciones de prueba por modalidad.
- Ingresa el nombre de la persona que recibe el equipo en el comercio y posibles observaciones.
- Presione **"Confirmar Instalación y Cerrar Gestión"**. La solicitud queda catalogada como **"POS Activo"** con registro de auditoría final.

Fase V: Auditoría Técnica, Casos Borde y Plan de Mejoras

1. Matriz de Vulnerabilidades y Bugs Identificados

ID	Severidad	Componente	Descripción del Hallazgo y Solución Requerida
BUG-01	CRÍTICO	DashboardView.jsx	Exportación CSV corrupta: Usa <code>.join('\\n')</code> con doble barra, imprimiendo el texto literal <code>\n</code> . Solución: Usar <code>\r\n</code> y anteponer BOM UTF-8 (<code>\uFEFF</code>).
BUG-02	CRÍTICO	Topbar.jsx	Fuga de Permisos en Móvil: El selector móvil permite a cualquier rol cambiarse a cualquier etapa. Solución: Aplicar filtro de roles por <code>currentUser.role</code> .
SEC-01	CRÍTICO	UsuarioController.java	Contraseñas en Texto Plano: Las claves viajan en JSON en <code>GET /api/usuarios</code> . Solución: Implementar <code>BCryptPasswordEncoder</code> y <code>@JsonProperty(WRITE_ONLY)</code> .
BUG-03	ALTO	Stage2Panel.jsx	Crash por Nulo: Falla si <code>documents</code> es indefinido en la solicitud. Solución: Usar encadenamiento opcional <code>request?.documents?.dpi?.status</code> .
ARC-01	ALTO	Afiliado.java	Cardinalidad Inconsistente (1:1 vs 1:N): Solo se guarda el primer afiliado en backend. Solución: Cambiar a relación <code>@ManyToOne</code> con <code>Solicitud</code> .
BUG-04	ALTO	Sidebar.jsx	Acción Destructiva sin Confirmar: "Restablecer datos demo" borra sin advertir. Solución: Añadir diálogo de confirmación modal.
BUG-05	MEDIO	workflow.js	Búsqueda sensible a acentos: Buscar "Optica" no encuentra "Óptica". Solución: Normalizar con <code>.normalize('NFD')</code> .
BUG-06	MEDIO	DocumentViewerModal.jsx	Previsualización rota de PDFs: Intenta abrir PDFs en etiquetas <code></code> . Solución: Utilizar etiquetas <code><iframe></code> o <code><embed></code> .

2. Plan de Acción y Roadmap de Mitigación

Prioridad P0 (Seguridad Inmediata)

1. Cifrado criptográfico de contraseñas de usuarios con **BCrypt** y ocultamiento del campo `password` en las respuestas JSON.
2. Incorporación de autenticación basada en **JWT (JSON Web Tokens)** y cierre de endpoints REST con Spring Security.
3. Restricción estricta de la política CORS al dominio del frontend.

Prioridad P1 (Integridad Transaccional y UI)

1. Normalización de la relación `Afiliado` a 1:N respecto a `Solicitud` para almacenar ventas múltiples.
2. Ampliación de la entidad `OrdenDespacho` con los campos de técnico asignado y dirección de entrega.
3. Corrección del generador CSV con saltos de línea estándar RFC 4180 y corrección de permisos en selector móvil.

3. Guía de Ejecución de Suites de Prueba Automatizadas

Para reproducir la totalidad de pruebas documentadas en este informe, ejecute los siguientes comandos en la terminal del proyecto:

```
# 1. Batería Formal de Estándares de Calidad (ISO 25010, OWASP, PCI-DSS, Unidad, Integración):  
node tests/standards-compliance-test.mjs  
  
# 2. Batería de Pruebas de Interfaz de Usuario y Componentes (30 elementos):  
node tests/ui-elements-test.mjs  
  
# 3. Simulación de Ciclo de Vida E2E y Sincronización REST (74 casos de prueba):  
node scratch/workflow-stages-test.mjs
```